

Deep Neural Networks

Assignment 4

From Perceptron to Backpropagation

Implement a neural network from scratch using NumPy and apply it to the Breast Cancer Wisconsin medical classification problem.

Programming environment: Python 3.x

Required libraries: NumPy and scikit-learn (dataset loading, splitting, and standardization only)

Visualization library: Matplotlib

Restriction: The learning algorithm must be implemented from scratch.

1 Assignment Overview

This assignment is designed as a progressive implementation exercise. You will first implement a single perceptron, observe its limitation on the XOR problem, and then build a small multilayer neural network capable of learning a non-linear decision boundary.

The assignment follows the conceptual progression of the lecture:

**Perceptron → Decision Boundary → XOR Limitation → Hidden Layer → Forward Pass
→ Loss → Backpropagation → Gradient Descent.**

The objective is not merely to obtain a high accuracy. You must be able to explain *why* the model makes a prediction, *how* the parameters are updated, and *why* adding a hidden layer changes the representational capability of the model.

2 Learning Outcomes

After completing this assignment, you should be able to:

1. Explain the operation of a perceptron.
2. Implement a perceptron from scratch using NumPy.
3. Explain weights, bias, threshold, learning rate, and epochs.
4. Understand and demonstrate linear separability.
5. Explain why a single perceptron cannot solve XOR.
6. Build a small multilayer perceptron (MLP).
7. Implement forward propagation using matrix operations.
8. Implement a loss/error function.
9. Derive and implement gradients using the chain rule.
10. Implement gradient descent parameter updates.
11. Compare activation functions experimentally.
12. Apply an MLP to the Breast Cancer Wisconsin medical dataset.
13. Evaluate a binary classifier using accuracy, precision, recall, F1-score, and a confusion matrix.
14. Visualize training loss and interpret the learning behavior.

3 Software and Restrictions

3.1 Required Software

Use:

- Python 3.x
- NumPy
- Matplotlib

You may use Jupyter Notebook, Google Colab, VS Code, PyCharm, or another Python environment.

3.2 Implementation Restrictions

Do not use TensorFlow, PyTorch, Keras, or a pre-built neural network classifier such as `MLPClassifier` for the main implementation. You may use `load_breast_cancer`, `train_test_split`, and `StandardScaler` for data preparation. The purpose of the assignment is to implement the learning mechanism yourself.

NumPy functions such as `dot`, matrix multiplication, `exp`, `zeros`, `maximum`, and array operations are allowed.

You may use Matplotlib for plots.

4 Part A: Perceptron From Scratch

4.1 Background

A perceptron first computes a weighted score:

$$z = \mathbf{w}^T \mathbf{x} + b.$$

For a binary step activation:

$$\hat{y} = \begin{cases} 1, & z \geq 0, \\ 0, & z < 0. \end{cases}$$

When the prediction is incorrect, update the parameters using:

$$\Delta w_i = \alpha(d - y)x_i,$$

$$\Delta b = \alpha(d - y),$$

where α is the learning rate, d is the desired target, and y is the current prediction.

4.2 Task A1 — Implement the Step Function

Implement:

```
1 import numpy as np
2
3 def unit_step(x):
4     # Return 1 when x >= 0, otherwise return 0
5     pass
```

Test the function with positive, negative, and zero inputs.

4.3 Task A2 — Implement a Perceptron Class

Complete the following class:

```
1 class Perceptron:
2
3     def __init__(self, learning_rate=0.01, epochs=100):
4         self.lr = learning_rate
5         self.epochs = epochs
6         self.weights = None
7         self.bias = None
8
9     def fit(self, X, y):
10         # 1. Initialize weights and bias
11         # 2. Repeat for the required number of epochs
12         # 3. Compute weighted sum
13         # 4. Compute prediction
```

```

14     # 5. Compute update
15     # 6. Update weights and bias
16     pass
17
18     def predict(self, X):
19         # Compute weighted sum and return class prediction
20         pass

```

Your implementation must:

- Initialize the weights according to the number of input features.
- Initialize the bias.
- Process every training example during an epoch.
- Calculate the prediction using the step function.
- Update the weights only when required by the perceptron rule.
- Update the bias consistently with the same error signal.

4.4 Task A3 — Train on AND

Use the following dataset:

x_1	x_2	d
0	0	0
0	1	0
1	0	0
1	1	1

Requirements:

1. Train your perceptron.
2. Print the final weights and bias.
3. Print predictions for all four samples.
4. Calculate classification accuracy.
5. Record the number of mistakes made in each epoch.

4.5 Task A4 — Decision Boundary

For a two-dimensional perceptron, plot:

- the four AND samples,
- Class 0 and Class 1 using different markers,
- the learned decision boundary.

Use the equation:

$$w_1x_1 + w_2x_2 + b = 0.$$

In your report, explain how changing w_1 , w_2 , or b affects the orientation or position of the boundary.

5 Part B: Linear Separability and XOR

5.1 Task B1 — XOR Dataset

Use:

x_1	x_2	d
0	0	0
0	1	1
1	0	1
1	1	0

Train the perceptron from Part A on this dataset.

5.2 Task B2 — Analyze the Failure

Answer the following:

1. Does the perceptron converge?
2. What accuracy does it achieve?
3. How does the number of mistakes change across epochs?
4. Plot the XOR samples.
5. Attempt to draw one straight decision boundary separating the two classes.
6. Explain why one straight line cannot separate the XOR classes.

Important concept: A single perceptron creates a linear decision boundary. XOR is not linearly separable. This limitation motivates the introduction of a hidden layer.

6 Part C: Breast Cancer Wisconsin Dataset

6.1 Dataset

For the real-world classification component, use the **Breast Cancer Wisconsin (Diagnostic) Dataset** available through `scikit-learn`. The dataset contains 569 samples, 30 numerical input features, and two target classes: malignant and benign.

Important: This is a binary-classification problem. The main neural network must therefore use a **single sigmoid output neuron**, rather than a Softmax output layer.

Load the dataset using:

```

1 from sklearn.datasets import load_breast_cancer
2
3 data = load_breast_cancer()
4 X = data.data
5 y = data.target
6
7 print("X shape:", X.shape)
8 print("y shape:", y.shape)
9 print("Classes:", data.target_names)
10 print("Features:", data.feature_names)
```

Verify that:

$$X \in \mathbb{R}^{569 \times 30}.$$

6.2 Task C1 — Explore the Dataset

Perform the following:

1. Print the number of samples and features.
2. Print all 30 feature names.
3. Print the two class names.
4. Count the number of samples in each class.
5. Check whether any missing values are present.
6. Calculate the mean and standard deviation of the input features.

Create a plot showing the number of samples in each class.

In your report, explain why feature scaling is important before training a neural network with gradient descent.

6.3 Task C2 — Train/Test Split and Standardization

Split the dataset into training and testing sets:

```

1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.20, random_state=42, stratify=y
6 )
7
8 scaler = StandardScaler()
9 X_train = scaler.fit_transform(X_train)
10 X_test = scaler.transform(X_test)

```

Requirements:

- Use 80% of the data for training and 20% for testing.
- Use stratification.
- Fit the scaler only on the training data.
- Transform the test data using the fitted scaler.

Do not fit the scaler separately on the test set. Doing so introduces information from the test data into preprocessing.

6.4 Network Architecture

Construct the following multilayer perceptron:

30 Inputs
 $\rightarrow$
16 Hidden Neurons
 $\rightarrow$
8 Hidden Neurons
 $\rightarrow$
1 Output Neuron

Thus:

$30 \rightarrow 16 \rightarrow 8 \rightarrow 1$

Use ReLU activation in the hidden layers:

$$\text{ReLU}(x) = \max(0, x),$$

and sigmoid activation in the output layer:

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

The forward pass is:

$$Z_1 = XW_1 + b_1, \quad A_1 = \text{ReLU}(Z_1),$$

$$Z_2 = A_1W_2 + b_2, \quad A_2 = \text{ReLU}(Z_2),$$

$$Z_3 = A_2W_3 + b_3, \quad \hat{Y} = \sigma(Z_3).$$

Parameter dimensions:

$$W_1 : 30 \times 16, \quad b_1 : 1 \times 16,$$

$$W_2 : 16 \times 8, \quad b_2 : 1 \times 8,$$

$$W_3 : 8 \times 1, \quad b_3 : 1 \times 1.$$

6.5 Task C3 — Initialize the Network

Use a reproducible random seed:

```

1 np.random.seed(42)
2
3 W1 = np.random.randn(30, 16) * 0.01
4 b1 = np.zeros((1, 16))
5
6 W2 = np.random.randn(16, 8) * 0.01
7 b2 = np.zeros((1, 8))
8
9 W3 = np.random.randn(8, 1) * 0.01
10 b3 = np.zeros((1, 1))

```

You may use another reasonable initialization, but document it.

6.6 Task C4 — Implement Forward Propagation

Implement:

```

1 def forward(X, W1, b1, W2, b2, W3, b3):
2     Z1 = X @ W1 + b1
3     A1 = relu(Z1)
4
5     Z2 = A1 @ W2 + b2
6     A2 = relu(Z2)
7
8     Z3 = A2 @ W3 + b3
9     A3 = sigmoid(Z3)
10
11     return A1, A2, A3

```

For at least one test sample, display:

$$X \rightarrow Z_1 \rightarrow A_1 \rightarrow Z_2 \rightarrow A_2 \rightarrow Z_3 \rightarrow A_3.$$

Explain the role of each intermediate quantity.

7 Part D: Binary Cross-Entropy Loss

7.1 Task D1 — Implement Binary Cross-Entropy

Because this is binary classification, use binary cross-entropy:

$$E = -\frac{1}{m} \sum_{i=1}^m [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)].$$

Implement a numerically safe version:

```

1 def binary_cross_entropy(y_true, y_pred):
2     eps = 1e-8
3     y_pred = np.clip(y_pred, eps, 1 - eps)
4     return -np.mean(
5         y_true * np.log(y_pred)
6         + (1 - y_true) * np.log(1 - y_pred)
7     )

```

Calculate the initial training loss before training.

7.2 Task D2 — Loss Curve

During training, store the loss after every epoch and plot:

Epoch vs. Binary Cross-Entropy Loss

Explain whether the loss decreases and what this indicates about learning.

8 Part E: Backpropagation

8.1 Task E1 — Understand the Chain Rule

For an output-layer weight, express the gradient using:

$$\frac{\partial E}{\partial w} = \frac{\partial E}{\partial \hat{Y}} \frac{\partial \hat{Y}}{\partial Z_3} \frac{\partial Z_3}{\partial w}.$$

Explain the meaning of each factor.

8.2 Task E2 — Output-Layer Gradient

For binary cross-entropy:

$$E = - \left[y \log(\hat{Y}) + (1 - y) \log(1 - \hat{Y}) \right]$$

and:

$$\hat{Y} = \sigma(Z_3),$$

derive:

$$\frac{\partial E}{\partial \hat{Y}}, \quad \frac{\partial \hat{Y}}{\partial Z_3}, \quad \frac{\partial E}{\partial Z_3}.$$

Show that:

$$\boxed{\frac{\partial E}{\partial Z_3} = \hat{Y} - y.}$$

Then derive the gradients for W_3 and b_3 .

8.3 Task E3 — Hidden-Layer Gradients

Using the chain rule, derive the gradients for both hidden layers.

Show the complete path:

$$W_1 \rightarrow Z_1 \rightarrow A_1 \rightarrow Z_2 \rightarrow A_2 \rightarrow Z_3 \rightarrow \hat{Y} \rightarrow E.$$

Derive:

$$\frac{\partial E}{\partial W_3}, \quad \frac{\partial E}{\partial b_3},$$

$$\frac{\partial E}{\partial W_2}, \quad \frac{\partial E}{\partial b_2},$$

and:

$$\frac{\partial E}{\partial W_1}, \quad \frac{\partial E}{\partial b_1}.$$

Use:

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0, \\ 0, & z \leq 0. \end{cases}$$

The derivation must show where the input matrix X appears.

8.4 Task E4 — Implement Backpropagation

Implement:

```

1 def backward(X, y, A1, A2, A3, Z1, Z2, W2, W3):
2     dZ3 = A3 - y
3
4     dW3 = A2.T @ dZ3 / len(X)
5     db3 = np.mean(dZ3, axis=0, keepdims=True)
6
7     dA2 = dZ3 @ W3.T
8     dZ2 = dA2 * relu_derivative(Z2)
9     dW2 = A1.T @ dZ2 / len(X)
10    db2 = np.mean(dZ2, axis=0, keepdims=True)
11
12    dA1 = dZ2 @ W2.T
13    dZ1 = dA1 * relu_derivative(Z1)
14    dW1 = X.T @ dZ1 / len(X)
15    db1 = np.mean(dZ1, axis=0, keepdims=True)
16
17    return dW1, db1, dW2, db2, dW3, db3

```

Do not use automatic differentiation.

9 Part F: Gradient Descent and Training

9.1 Task F1 — Parameter Update

Use:

$$W_{\text{new}} = W_{\text{old}} - \eta \frac{\partial E}{\partial W},$$

$$b_{\text{new}} = b_{\text{old}} - \eta \frac{\partial E}{\partial b}.$$

Update all six parameter sets:

$$W_1, b_1, W_2, b_2, W_3, b_3.$$

9.2 Task F2 — Complete Training Loop

Combine forward propagation, binary cross-entropy, backpropagation, and gradient descent into one training procedure.

Follow these steps:

1. Initialize parameters.
2. Perform a forward pass on the training data.
3. Calculate binary cross-entropy.
4. Calculate gradients using backpropagation.
5. Update all weights and biases.
6. Store the loss.
7. Repeat for the required number of epochs.

A suggested interface is:

```
1 def train(X, y, learning_rate=0.01, epochs=2000):
2     # initialize parameters
3
4     for epoch in range(epochs):
5         # forward propagation
6         # binary cross-entropy
7         # backpropagation
8         # gradient descent
9         pass
10
11     return parameters, loss_history
```

9.3 Task F3 — Train the Breast Cancer Classifier

Train the $30 \rightarrow 16 \rightarrow 8 \rightarrow 1$ MLP using the training data and evaluate it on the held-out test set.

Report:

- final training loss,
- final test loss,
- training accuracy,
- test accuracy,
- precision,
- recall,
- F1-score,
- number of training epochs.

Convert probabilities to classes using:

$$\hat{y}_{class} = \begin{cases} 1, & \hat{y} \geq 0.5, \\ 0, & \hat{y} < 0.5. \end{cases}$$

Create a confusion matrix and identify:

- True Positive (TP),
- True Negative (TN),
- False Positive (FP),
- False Negative (FN).

Discuss why accuracy alone may not be sufficient when evaluating a medical classification model.

10 Part G: Comprehensive Activation Function Analysis

In this part, you will experimentally investigate the performance of the activation functions on the **Breast Cancer Wisconsin dataset** and determine which activation function provides the best overall performance.

The network architecture must remain fixed throughout the experiment:

$$30 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

The **output layer must always use Sigmoid**. Only the activation function of the two hidden layers should be changed.

Fair comparison: For every activation function, keep the following identical: training/test split, preprocessing, network architecture, weight initialization, learning rate, number of epochs, loss function, batch strategy, and classification threshold.

10.1 Task G1 — Implement Activation Functions

Implement the following activation functions from scratch using NumPy:

1. Sigmoid
2. Tanh
3. ReLU
4. Leaky ReLU
5. PReLU
6. ELU
7. SELU
8. Softplus
9. GELU
10. SiLU/Swish
11. Mish

For each activation function, implement both the activation function and its derivative where required for backpropagation.

For example:

$$\sigma(x) = \frac{1}{1 + e^{-x}},$$

$$\text{ReLU}(x) = \max(0, x),$$

$$\text{LeakyReLU}(x) = \begin{cases} x, & x > 0, \\ \alpha x, & x \leq 0. \end{cases}$$

Use a fixed value of α for Leaky ReLU and document the value used. Also implement the derivatives required by your training algorithm.

10.2 Task G2 — Verify Activation Functions

Generate a range of input values, for example:

$$x \in [-5, 5],$$

and plot all activation functions.

Create a separate plot for the derivatives.

Your plots should allow comparison of:

- output range,
- saturation behavior,
- behavior for negative inputs,
- behavior for positive inputs,
- gradient behavior.

In your report, explain how these properties may influence neural-network optimization.

10.3 Task G3 — Train the Network with Each Activation

Train the same:

$$30 \rightarrow 16 \rightarrow 8 \rightarrow 1$$

MLP using each of the following hidden-layer activation functions:

Sigmoid
Tanh
ReLU
Leaky ReLU
PReLU
ELU
SELU
Softplus
GELU
SiLU/Swish
Mish

For every experiment:

- Use the same training and test sets.
- Use the same standardized features.
- Use the same initial weights.
- Use the same learning rate.
- Use the same number of epochs.
- Use binary cross-entropy loss.
- Use Sigmoid in the output layer.
- Use 0.5 as the classification threshold.

Record the following for every activation:

- final training loss,
- final test loss,
- training accuracy,
- test accuracy,
- precision,
- recall,
- F1-score,
- number of epochs,
- training time, if measured.

10.4 Task G4 — Activation Performance Comparison

Create the following table:

Activation	Train Loss	Test Loss	Accuracy	Precision	Recall	F1
Sigmoid						
Tanh						
ReLU						
Leaky ReLU						
PReLU						
ELU						
SELU						
Softplus						
GELU						
SiLU/Swish						
Mish						

Create bar plots comparing:

1. Test accuracy
2. Precision
3. Recall
4. F1-score
5. Final test loss

10.5 Task G5 — Training Convergence Analysis

Plot the training loss against epoch for all activation functions.

Analyze:

1. Which activation converges fastest?
2. Which activation reaches the lowest training loss?
3. Which activation achieves the lowest test loss?
4. Which activation achieves the highest test accuracy?
5. Which activation achieves the highest F1-score?
6. Which activation shows unstable or slow convergence?

Discuss whether the activation that obtains the lowest training loss also provides the best test performance.

10.6 Task G6 — Multiple Random Initializations

A single random initialization may not provide a reliable comparison. Therefore, repeat the experiment using at least five different random seeds:

$\{42, 52, 62, 72, 82\}$.

For each activation function, calculate:

Mean Accuracy

and

Standard Deviation of Accuracy.

Also calculate the mean and standard deviation of the F1-score.

Create a summary table:

Activation	Mean Accuracy	Std. Accuracy	Mean F1	Std. F1
Sigmoid				
Tanh				
ReLU				
Leaky ReLU				
PReLU				
ELU				
SELU				
Softplus				
GELU				
SiLU/Swish				
Mish				

The purpose of multiple runs is to determine whether the observed performance is reproducible or is mainly caused by a particular random initialization.

10.7 Task G7 — Determine the Best Activation Function

Based on your experiments, determine the best-performing activation function.

Do not select the best activation using accuracy alone.

Consider:

Accuracy, Precision, Recall, F1-score, Test Loss

and the variability across different random seeds.

For the medical classification problem, pay particular attention to **Recall**, since missing a positive case can be more important than achieving a small improvement in overall accuracy.

Answer:

1. Which activation function achieved the highest mean accuracy?
2. Which achieved the highest mean F1-score?
3. Which achieved the highest recall?
4. Which achieved the lowest mean test loss?
5. Which activation was the most stable across random seeds?
6. Which activation would you select for this medical classification problem?
7. Does the same activation win under every evaluation metric?
8. What trade-offs exist between accuracy, recall, F1-score, and convergence speed?

Finally, provide a clear conclusion:

Best Activation Function:

Best Activation Function:

Best Activation Function:

Best Activation Function:

Justify your choice using the experimental results rather than theoretical expectations alone.

10.8 Task G8 — Final Activation Analysis

Write a short analysis of approximately 300–500 words discussing the results.

Your analysis should explain:

- why different activation functions produce different optimization behavior;
- which functions suffered from saturation or weak gradients;
- whether ReLU-based functions performed better than Sigmoid/Tanh;
- whether smooth activations such as GELU, SiLU/Swish, or Mish showed any advantage;
- whether the best activation was consistent across random seeds;
- which activation you would recommend for this dataset and why.

Your conclusion must be based on the experimental evidence collected in this assignment.

11 Part H: Dying ReLU Experiment

11.1 Task H1 — Investigate Dead Neurons

For a ReLU neuron:

$$h = \text{ReLU}(z), \quad z = w^T x + b.$$

Using the standardized Breast Cancer Wisconsin training data, investigate the effect of making a hidden-layer bias sufficiently negative.

Calculate:

$$\text{ReLU}'(z).$$

Report the fraction of activations that become zero.

Answer:

1. What happens when $z < 0$?
2. What is the ReLU derivative in this region?
3. What happens to the gradient?
4. Can the neuron easily recover?
5. How could Leaky ReLU help?

12 Part I: Matrix Implementation

12.1 Task I1 — Replace Scalar Calculations

Implement the complete network using matrix operations:

$$Z_1 = XW_1 + b_1, \quad A_1 = f(Z_1),$$

$$Z_2 = A_1W_2 + b_2, \quad A_2 = f(Z_2),$$

$$Z_3 = A_2W_3 + b_3, \quad A_3 = \sigma(Z_3).$$

Explain why matrix multiplication is useful when processing many samples and neurons.

12.2 Task I2 — Verify Your Implementation

For at least one sample, compare scalar and matrix-based forward-pass calculations and show that the results agree up to numerical precision.

Also verify the dimensions of every intermediate matrix.

13 Part J: Learning-Rate Experiment

13.1 Task J1

Train the same Breast Cancer Wisconsin MLP using:

$$\eta \in \{0.001, 0.01, 0.05, 0.1\}.$$

Keep the same initialization, split, and number of epochs for a fair comparison.

Record:

Learning Rate	Epochs	Test Loss	Accuracy	F1
0.001				
0.01				
0.05				
0.1				

Plot the training-loss curves and discuss convergence, stability, and final test performance.

14 Part K: Final Integrated Program

Combine your work into one clean Python program for the Breast Cancer Wisconsin dataset.

Your final program should contain at least:

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 from sklearn.datasets import load_breast_cancer
5 from sklearn.model_selection import train_test_split
6 from sklearn.preprocessing import StandardScaler
7
8 def sigmoid(x):
9     pass
10
11 def sigmoid_derivative(x):
12     pass
13
14 def relu(x):
15     pass
16
17 def relu_derivative(x):
18     pass
19
20 def binary_cross_entropy(y_true, y_pred):
21     pass
22
23 def initialize_parameters():
24     pass
25
26 def forward(X, parameters):
27     pass
28
29 def backward(X, y, cache, parameters):
30     pass
31
32 def update_parameters(parameters, gradients, learning_rate):
33     pass
34
35 def train(X, y, learning_rate, epochs):
36     pass
37
38 def predict(X, parameters):
39     pass

```

The final program must:

1. Load the Breast Cancer Wisconsin dataset.
2. Split the data into training and testing sets.
3. Standardize features using the training set.

4. Train the $30 \rightarrow 16 \rightarrow 8 \rightarrow 1$ MLP.
5. Use ReLU in the hidden layers.
6. Use sigmoid in the output layer.
7. Use binary cross-entropy loss.
8. Perform backpropagation manually.
9. Perform gradient-descent updates manually.
10. Print training and test performance.
11. Print accuracy, precision, recall, and F1-score.
12. Display a confusion matrix.
13. Plot the training-loss curve.

14.1 Expected Final Evaluation

Your final output should include:

```
1 ===== BREAST CANCER WISCONSIN: MLP =====
2 Training samples:
3 Testing samples:
4 Number of features: 30
5
6 ===== TRAINING =====
7 Final training loss:
8 Epochs:
9
10 ===== TEST RESULTS =====
11 Test loss:
12 Accuracy:
13 Precision:
14 Recall:
15 F1-score:
16
17 Confusion Matrix:
18 [[TN FP]
19  [FN TP]]
```

Do not assume a particular accuracy in advance. Report the value obtained from your own implementation and explain the result.

15 Report Requirements

Submit your Python notebook/script.

The report should contain:

1. **Perceptron Implementation** — Explain your algorithm.
2. **AND Experiment** — Results and decision boundary.
3. **XOR Experiment** — Demonstrate perceptron failure.
4. **Dataset Exploration** — Describe the Breast Cancer Wisconsin dataset.
5. **Preprocessing** — Explain train/test splitting and standardization.

6. **MLP Architecture** — Explain the $30 \rightarrow 16 \rightarrow 8 \rightarrow 1$ network.
7. **Forward Propagation** — Show equations and one numerical example.
8. **Binary Cross-Entropy** — Define and explain the selected loss.
9. **Backpropagation** — Show the chain-rule derivation.
10. **Gradient Descent** — Explain parameter updates.
11. **Activation Experiment** — Compare Sigmoid, Tanh, and ReLU.
12. **Learning-Rate Experiment** — Include table and plots.
13. **Medical Classification Evaluation** — Discuss accuracy, precision, recall, F1-score, and the confusion matrix.

16 Required Figures

Your final submission must contain at least the following figures:

1. AND dataset with learned decision boundary.
2. XOR dataset showing the two classes.
3. Breast Cancer Wisconsin class distribution.
4. Training loss curve for the MLP.
5. Comparison of activation functions.
6. Comparison of learning rates.
7. Confusion matrix for the final test set.

Every figure must have:

- a figure number,
- a descriptive caption,
- labeled axes where applicable,
- readable text.

17 Expected Output Summary

At minimum, your final notebook/script should display:

```
1 ===== PERCEPTRON: AND =====
2 Final weights:
3 Final bias:
4 Predictions:
5 Accuracy:
6
7 ===== PERCEPTRON: XOR =====
8 Final weights:
9 Final bias:
10 Predictions:
11 Accuracy:
12 Number of mistakes per epoch:
13
```

```

14 ===== MLP: BREAST CANCER WISCONSIN =====
15 Training samples:
16 Testing samples:
17 Number of features: 30
18 Final training loss:
19 Final test loss:
20 Accuracy:
21 Precision:
22 Recall:
23 F1-score:
24 Epochs:
25
26 Confusion Matrix:
27 [[TN FP]
28  [FN TP]]
29
30 ===== ACTIVATION COMPARISON =====
31 Sigmoid:
32 Tanh:
33 ReLU:
34
35 ===== LEARNING RATE COMPARISON =====
36 0.001:
37 0.01:
38 0.05:
39 0.1:

```

18 Optional Challenge Tasks

These tasks are optional and can be used for additional credit.

Challenge 1 — Softmax Output

Implement:

$$p_i = \frac{e^{z_i}}{\sum_{j=1}^K e^{z_j}}$$

and verify:

$$0 \leq p_i \leq 1, \quad \sum_i p_i = 1.$$

Explain why Softmax is useful for multiclass output.

Challenge 2 — Leaky ReLU

Implement:

$$f(x) = \begin{cases} x, & x > 0, \\ \alpha x, & x \leq 0. \end{cases}$$

Compare its behavior with ReLU.

Challenge 3 — Mini-Batch Training

Modify your training procedure so that it can process a batch of samples rather than updating using the complete dataset at once.

Challenge 4 — Decision Surface

Create a larger synthetic two-dimensional dataset and visualize the decision regions produced by the trained MLP.